When Recall Stops Discriminating: Cost-Adjusted Model Selection for Failure Detection in Industrial Telemetry on NASA C-MAPSS FD001

Puru Pandey
Independent Researcher
Greater Noida, Uttar Pradesh, India
purupandey2001@gmail.com
github.com/Puru2001pandey/industrial-telemetry-analytics-research

September 2026

Abstract

Binary failure classification in predictive maintenance requires detecting at-risk machine states before failure occurs. Model selection on this task is usually driven by a single metric—recall, or an aggregate such as weighted F1—and this paper shows, on the NASA C-MAPSS FD001 benchmark (20,631 cycles, 100 engine units), that neither metric is sufficient to identify the model a practitioner should deploy. We evaluate five approaches: logistic regression on a single rolling degraded-state counter derived from per-unit-normalised sensor composites; a 100-tree Random Forest on all 21 raw sensor channels; and three simple thresholding baselines. On a held-out seed-42 unit-level split, **four of the five models achieve at-risk recall between 0.998 and 1.000, yet their total operational costs range from $2.55M to $5.29M**—a 2.07$\times$ spread that recall cannot detect, driven entirely by precision (0.370–0.549). The Random Forest, meanwhile, attains the highest weighted F1 (0.957) while costing 56% more than the cheapest model. Selecting on either metric alone therefore picks the wrong model. To estimate generalisation honestly we adopt a nested unit-level protocol: 60 training, 20 validation and 20 test engines per split, with the degradation threshold fitted on training-engine healthy cycles only (RUL > 40) and (w, τ) chosen on validation engines. Across 30 such splits the rolling-count model costs $2.11M $\pm$ $0.39M versus $3.84M $\pm$ $0.95M for the Random Forest, winning on cost in **30 of 30 splits** at roughly 40% of the Random Forest's variance. We further quantify two selection effects that are rarely reported on this benchmark: choosing (w, τ) by minimising cost on a single test split understates true cost by **$790,167** (42%), and per-split tuning on 20 validation engines is *worse* than a fixed setting ($2.11M vs. $1.89M), because the optimum is not identifiable from that sample—12 distinct (w, τ) pairs are selected across 30 splits, the modal choice appearing in only 5. This version also corrects a degradation-direction error in the sensor preprocessing present in versions 1–3 (Section 4.1). Code is available at `https://github.com/Puru2001pandey/industrial-telemetry-analytics-research`.

Keywords: predictive maintenance, feature engineering, C-MAPSS, logistic regression, random forest, cost-sensitive evaluation, class imbalance, industrial IoT, leak-free evaluation, nested cross-validation, selection bias.

1 Introduction

Predictive maintenance (PdM) systems aim to detect incipient failures in industrial equipment before they occur [1]. The NASA C-MAPSS benchmark [2] has become a standard testbed for prognostic methods, with most prior work focusing on Remaining Useful Life (RUL) regression [4]. However, real-world maintenance decisions often require a binary alert: will this machine fail within the next N cycles?

For this binary task, the cost structure is asymmetric. A missed failure—a false negative—leads to unplanned downtime, safety incidents, and consequential damage costs that dwarf the cost of a scheduled inspection. A false alarm—a false positive—triggers an unnecessary inspection and minor operational disruption. The standard aggregate evaluation metric, weighted F1, treats these two error types symmetrically and thus may favour models that are suboptimal for deployment. A less obvious pitfall, which this corrected analysis surfaces directly, is that *recall alone* is also an incomplete criterion: a model can maximise recall by flagging almost everything, at the cost of a false-alarm rate that makes it operationally unusable.

This paper set out to test whether a single, domain-informed temporal feature could match a high-dimensional sensor ensemble on C-MAPSS FD001. The question it ended up answering is a different and more general one: which metric should decide that comparison at all.

Two corrections separate this version from its predecessors, and both are described in full rather than folded silently into the

results. Versions 1–2 estimated the degradation percentile over each engine's complete run-to-failure trajectory, a look-ahead bias corrected in version 3 by restricting the estimate to healthy-phase cycles (RUL > 40) of training engines. Versions 1–3 additionally assigned three of fourteen sensors to the wrong side of the degradation composite (Section 4.1), which this version corrects and which required re-running every experiment.

Under the corrected pipeline, four of the five models evaluated reach at-risk recall between 0.9984 and 1.0000 on the held-out engines, while their operational costs span \$2.55M to \$5.29M. Recall cannot separate them. Weighted F1 ranks first the one model that misses 70 failures. The distinction this paper draws—between a model that scores best on a metric in isolation and a model that is the right deployment choice under a stated cost structure—is therefore not a matter of emphasis but of arriving at different answers.

We estimate generalisation under a nested unit-level protocol rather than a single split, and quantify two selection effects that this exercise made visible: a \$790,167 optimism from choosing hyperparameters on the test engines, and the finding that per-split tuning on a small validation set does not beat a fixed default.

2 Related Work

The C-MAPSS benchmark [2] has attracted a large body of work, most of it aimed at RUL regression. Li et al. [4] achieve strong RMSE on FD001 with deep convolutional networks, and this line of work has since been extended with attention-based and sequence-model architectures. These are impressive results, but they address a different question—predicting how many cycles remain—rather than the binary question of whether a failure is imminent.

Khelif et al. [5] take the opposite approach to the one adopted here: they estimate RUL directly from sensor values using support vector regression, explicitly avoiding the intermediate steps of constructing a health indicator and setting a failure threshold. That design choice is instructive precisely because this paper does the reverse—it builds a single domain-informed health indicator and thresholds it—and because their target is continuous RUL rather than a binary at-risk alert. Neither their work nor the regression literature more broadly evaluates at-risk recall against an explicit deployment cost model, which is the gap this paper targets directly. Ramasso and Saxena [3] make a point that shaped the preprocessing decisions in this paper: raw C-MAPSS sensor readings carry substantial engine-specific baseline offsets, so any meaningful comparison requires per-unit normalisation before modelling. Without it, differences between engines swamp the degradation signal.

Class-imbalanced classification more broadly has a well-established toolkit, including reweighting the training loss so that easy, well-classified majority examples contribute less to the objective and training concentrates on the harder minority cases [6]. Most treatments of this kind optimise an aggregate loss or metric (weighted accuracy, AUC, or F1) rather than isolating recall on the minority class as the deployment-critical quantity, and fewer still evaluate the resulting recall/precision trade-off against an explicit monetary cost model, as this paper does in Section 5.

The literature lacks a direct, cleanly-controlled comparison between a single domain-informed feature and a full-sensor ensemble evaluated specifically on at-risk recall and operational cost rather than RMSE, accuracy, or overall weighted F1. Papers that adopt simpler features tend to compare against other simple features; papers that use ensemble methods tend to optimise aggregate metrics. The narrow question this paper poses—does the model with the best recall in isolation also have the best deployment cost, once precision is accounted for?—does not appear to have been posed directly in prior work on this benchmark. That gap motivates this study.

3 Dataset

NASA C-MAPSS FD001. The Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset [2] provides run-to-failure telemetry for turbofan engines simulated under a single operating condition (FD001 subset). Each row records one operational cycle for one engine unit: three operating-setting columns and 21 sensor readings. The training file contains 20,631 rows across 100 engine units, with per-unit cycle counts ranging from 128 to 362 (mean 206). No holdout test file is used; evaluation is performed on a held-out partition of training engines.

Binary label definition. For engine u, let $C_u^{\max}$ denote its maximum recorded cycle (the failure point). The Remaining Useful Life at cycle c is $\mathrm{RUL}(u, c) = C_u^{\max} - c$. We define:

$$y_{u,c} = \begin{cases} 1 & \text{if } \mathrm{RUL}(u,c) \le 30 \quad \text{(at risk)} \\ 0 & \text{otherwise} \quad \text{(healthy)} \end{cases} \tag{1}$$

This yields 17,531 healthy and 3,100 at-risk observations (15.0% minority class).

4 Methodology

4.1 Sensor Preprocessing

Several C-MAPSS sensors are constant or near-constant across FD001 and provide no discriminative signal [3]. We retain 14 sensors with meaningful variance: $\{s_{02}, s_{03}, s_{04}, s_{07}, s_{08}, s_{09}, s_{11}, s_{12}, s_{13}, s_{14}, s_{15}, s_{17}, s_{20}, s_{21}\}$, categorised by degradation direction: rising sensors $\mathcal{R} = \{s_{02}, s_{03}, s_{04}, s_{08}, s_{09}, s_{11}, s_{13}, s_{14}, s_{15}, s_{17}\}$ (values increase with degradation) and falling sensors $\mathcal{F} = \{s_{07}, s_{12}, s_{20}, s_{21}\}$ (values decrease).

Correction to the direction assignment. In versions 1–3 of this record, s_{07} and s_{12} were assigned to $\mathcal{R}$ and s_{17} to

$\mathcal{F}$. Verifying the assignment against the data shows all three were placed on the wrong side: the Pearson correlation of each normalised sensor with RUL is $+0.657$ for s_{07} and $+0.672$ for s_{12} (both *decrease* as the engine degrades) and -0.606 for s_{17} (it *increases*). Under Equation 3 this made both terms of the composite partially self-cancelling, since two of eleven rising sensors and one of only three falling sensors moved against the degradation signal. The assignment above is the corrected one, and all results in this version use it.

Per-unit min-max normalisation removes engine-specific baseline offsets:

$$\tilde{x}_{u,c,s} = \frac{x_{u,c,s} - \min_c x_{u,c,s}}{\max_c x_{u,c,s} - \min_c x_{u,c,s} + \varepsilon} \tag{2}$$

where $\varepsilon = 10^{-9}$ prevents division by zero.

4.2 Degradation Score and Leak-Free Threshold

We define a composite degradation score for each cycle:

$$d_{u,c} = \frac{\frac{1}{|\mathcal{R}|}\sum_{s\in\mathcal{R}} \tilde{x}_{u,c,s} + \left(1 - \frac{1}{|\mathcal{F}|}\sum_{s\in\mathcal{F}} \tilde{x}_{u,c,s}\right)}{2} \tag{3}$$

Critical correction: leak-free threshold estimation. In versions 1 and 2 of this record, a cycle was labelled degraded if its score exceeded the 70th percentile of that engine's *full* score distribution—including at-risk cycles, introducing a look-ahead bias. In this corrected version, the threshold is estimated *strictly from healthy-phase cycles only*: for each training engine, we pool cycles with $\mathrm{RUL} > 30 + 10$ (where the 10-cycle buffer avoids bleeding into the at-risk window) and set the global threshold as the 70th percentile of these pooled healthy scores. This single threshold is then applied to all cycles (train and test) at inference time. We also evaluate a per-engine variant where each engine is calibrated using only its own healthy-phase data (more realistic for deployment with burn-in periods); results are reported in Section 5.

A cycle is labelled *degraded* if its score exceeds this leak-free threshold:

$$g_{u,c} = \mathbf{1}[d_{u,c} \geq \hat{\tau}] \tag{4}$$

$$\hat{\tau} = Q_{0.70}\big(\{d_{u,c} \mid \text{training unit, RUL} > 40\}\big) \tag{5}$$

The **health-window feature** is the count of degraded cycles in the 10-cycle rolling window ending at cycle c:

$$h_{u,c} = \sum_{k=\max(1,\, c-9)}^{c} g_{u,k} \tag{6}$$

This scalar encodes the recent degradation trajectory as a single interpretable value.

4.3 Model A: Logistic Regression

We train ℓ_2-regularised logistic regression on the scalar $h_{u,c}$ with class weights inversely proportional to class frequency (`balanced`). This 1-feature model has a trivially interpretable decision rule.

4.4 Model B: Random Forest Baseline

A 100-tree Random Forest is trained on all 21 *raw* sensor columns after applying a global `StandardScaler` (`class_weight='balanced'`, seed 42). The RF intentionally receives globally standardised rather than per-unit normalised inputs to reflect a realistic high-dimensional baseline: a practitioner would apply StandardScaler before training a Random Forest without the domain knowledge needed to choose the 14 informative sensors or compute the per-unit normalisation.

4.5 Additional Baselines

To understand whether the rolling-count feature specifically drives its result, and to test whether simpler alternatives dominate it, we include three baselines that were absent from versions 1 and 2:

(a) **Single best sensor:** threshold the normalised sensor with the highest absolute correlation with the at-risk label on training data.

(b) **Raw composite score:** threshold $d_{u,c}$ directly (no rolling window).

(c) **Moving average:** threshold the w-cycle moving average of $d_{u,c}$.

As Section 5 shows, these baselines match the rolling-count feature on recall almost exactly; what separates them is precision, and therefore operational cost.

4.6 Evaluation Protocol

Unit-level split. 80 engine units are used for training and 20 for testing (seed 42 shuffle). All cycles from a given engine appear in exactly one partition, eliminating temporal leakage from cycle-level random splits.

Primary metric. We report at-risk recall, precision, and weighted F1 for every model, but treat none of them in isolation as the deciding criterion. Instead, we compute total expected operational cost under a stated false-negative/false-positive cost ratio (Section 5) as the primary basis for model comparison, since this is the quantity a deployment decision actually depends on.

Nested protocol for generalisation estimates. Reporting cost at a (w, τ) chosen by minimising cost on the test engines would be selection bias, and Section 6.4 shows the resulting figure is optimistic by a wide margin. For the generalisation estimate in Table 5 we therefore partition the 100 engines three ways per seed—60 training, 20 validation, 20 test, always at engine level—fit the leak-free threshold on training-engine healthy cycles only, select (w, τ) by minimum cost on the *validation* engines, and evaluate once on the test engines. This is repeated for seeds 0–29.

Statistical stability. The 80/20 experiment is additionally repeated across 30 random splits (seeds 0–29) to assess robustness of the single-split comparison.

5 Results

Table 1 summarises performance on 3,852 test cycles from 20 held-out engine units (seed 42), using the leak-free global threshold.

Table 1 contains the paper's central observation. Four of the five models—the rolling-count logistic regression and all three baselines—achieve at-risk recall between 0.9984 and 1.0000. On recall they are indistinguishable. Their total operational costs, however, range from \$2.55M to \$5.29M, a factor of 2.07. A practitioner selecting on recall has no basis for choosing between them, and a one-in-four chance of picking the most expensive.

What separates them is precision, which ranges from 0.3695 to 0.5487 across those four models and determines the false-alarm volume that dominates the cost calculation once misses are near zero.

Weighted F1 fails differently. The Random Forest is the best model on both precision (0.8488) and weighted F1 (0.9568), and is the only model whose recall falls materially below 1.0 (0.8871). Selecting on weighted F1 picks the Random Forest and pays \$3.99M, 56% more than the \$2.55M of the model with the lowest cost. The aggregate metric is dominated by the 3,232 healthy cycles and is largely insensitive to the 70 missed failures that drive the cost.

Confusion matrices (seed 42, leak-free global threshold). At this setting the rolling-count model catches every at-risk cycle, and the entirety of its \$2.55M cost is false alarms: 510 unnecessary inspections. The three baselines catch essentially every failure too, but raise 918–1,058 false alarms to do it. The Random Forest sits at the other extreme, raising only 98 false alarms but missing 70 failures, which under this cost ratio is the more expensive error.

We note that $\tau = 70$ is inherited from earlier versions of this record and is not a tuned choice; Section 5.2 shows the cost surface and Section 6.4 explains why we do not select an operating point from it.

5.1 Statistical Stability Across 30 Splits

To assess whether the single-split comparison depends on the specific 80/20 partition, we repeated it with 30 random seeds using the *per-engine leak-free threshold* (each engine calibrated on its own healthy-phase data).

The LR attains recall 1.0 on every one of the 30 splits. We report the paired t-test for continuity with earlier versions of this record, but it should not be read as a meaningful significance test: with zero variance in one arm, the statistic reduces to asking whether the RF mean differs from a constant, and the t and p values are artefacts of that degeneracy rather than evidence about a contested comparison.

The zero variance is itself the finding. A classifier that never misses on any split, at precision 0.5487, is not a well-calibrated detector—it is one that flags a large share of all cycles. This is the same failure mode the paper attributes to the naive baselines, and it indicates that $\tau = 70$ is a poor operating point for the corrected composite. Section 5.2 shows the cost surface this sits on.

5.2 Sensitivity Analysis

The window length w and the degradation percentile threshold τ are design choices. To characterise how the operating point affects deployment cost, we vary $w \in \{5, 10, 15, 20\}$ and $\tau \in \{50, 60, 70, 80, 90\}$ using the leak-free global threshold and report total cost on the seed-42 test engines.

Two things follow. First, the LR is cheaper than the Random Forest in 17 of these 20 cells but achieves higher at-risk recall in all 20—the two criteria disagree on three settings, which is a direct instance of the paper's central point measured within a single model family.

Second, and more importantly, the minimum of this surface must not be used to choose an operating point, because every cell in it was computed on the test engines. Section 6.4 quantifies what that would cost.

5.3 Nested generalisation estimate

Table 5 reports the nested protocol of Section 4: (w, τ) selected on 20 validation engines, evaluated once on 20 disjoint test engines, repeated over 30 seeds.

Under honest selection the rolling-count model costs \$1.73M less per deployment on average and wins on cost in every split, at roughly 40% of the Random Forest's standard deviation. Its recall (0.9764 ± 0.0164) is no longer degenerate, which confirms that the zero-variance result in Table 3 was an artefact of the fixed $\tau = 70$ rather than a property of the feature.

Table 1: Classification on NASA C-MAPSS FD001 (20 held-out engines, 3,852 test cycles; $w = 10$, $\tau = 70$, leak-free global threshold). At-risk support: 620 cycles. **Bold**: lowest-cost model. *Italic*: best on that metric. Four models tie or near-tie on recall (0.9984–1.0000) while their costs span 2.07×.

Model	At-Risk Recall	Precision	Weighted F1	Total Cost*
LR – Rolling Degraded Count (1 feat.)	1.0000	0.5487	0.8812	**$2.55M**
RF – 21 Raw Sensors	0.8871	*0.8488*	*0.9568*	$3.99M
Baseline 1: Single Best Sensor (s_{11})	0.9984	0.3874	0.7790	$4.95M
Baseline 2: Raw Composite Score	1.0000	0.3695	0.7617	$5.29M
Baseline 3: Moving Average ($w = 10$)	1.0000	0.4031	0.7926	$4.59M

*Cost model: Missed failure (FN) = $50,000; False alarm (FP) = $5,000. Costs computed on the 3,852 test cycles.

Table 2: Confusion matrices on 3,852 test cycles ($w = 10$, $\tau = 70$). The LR misses no at-risk cycles but raises 510 false alarms; the RF misses 70 while raising 98.

	LR		RF	
	Pred. 0	Pred. 1	Pred. 0	Pred. 1
Actual 0	2,722	510	3,134	98
Actual 1	0	620	70	550

Table 3: 30-seed cross-validation stability (per-engine leak-free threshold, $w = 10, \tau = 70$).

Metric	LR (Rolling Count)	RF (21 Sensors)
Mean Recall	1.0000 ± 0.0000	0.8977 ± 0.0284
Range	1.0000 – 1.0000	0.8403 – 0.9581
Win Rate	30 / 30 (100%)	—
Paired t-test	$t = 19.38, \quad p = 3.84 \times 10^{-18}$	

Table 4: Total operational cost ($M) on the seed-42 test engines across the window/threshold grid. The Random Forest is constant at $3.99M because it uses neither w nor τ. Costs fall monotonically in τ for every w, and the minimum in this range sits at the grid edge.

w	$\tau = 50$	$\tau = 60$	$\tau = 70$	$\tau = 80$	$\tau = 90$
5	5.61	4.37	3.21	2.10	1.25
10	4.56	3.52	2.55	1.66	1.54
15	3.98	3.07	2.22	1.68	1.83
20	3.45	2.81	2.38	1.66	2.00

Table 5: Nested 60/20/20 unit-level protocol, 30 seeds. (w, τ) chosen by minimum validation cost; metrics reported on held-out test engines.

Metric (test engines)	LR (Rolling Count)	RF (21 Sensors)
Mean total cost	**$2.11M** $\pm$ $0.39M	$3.84M $\pm$ $0.95M
Cost range	$1.46M – $3.19M	$2.50M – $6.67M
Mean recall	0.9764 ± 0.0164	0.8982 ± 0.0372
Mean precision	0.6939 ± 0.0636	0.8074 ± 0.0555
Cost win rate	**30 / 30** (100%)	—

6 Discussion

6.1 Recall alone is not sufficient

Recall fails as a selection criterion here not because it ranks the models wrongly, but because it stops ranking them at all. Four of the five models in Table 1 lie between 0.9984 and 1.0000 at-risk recall—a spread of one missed cycle out of 620—while their costs differ by $2.74M. Recall has no resolving power in that region, and a practitioner using it would be choosing at random among four models whose economics differ by a factor of two.

This is a stronger statement than the one made in versions 1–3 of this record, which argued that simple baselines *out-recalled* the proposed feature. After the degradation-direction correction of Section 4.1 that is no longer true; nothing out-recalls the rolling-count model. The failure of recall as a criterion does not depend on it, and the corrected result isolates the mechanism more cleanly: once misses approach zero, all of the remaining cost variation lives in precision, which recall does not observe.

Weighted F1 fails for a different reason. It is computed over 3,852 cycles of which 3,232 are healthy, so it is dominated by majority-class performance. The Random Forest leads on it (0.9568) while missing 70 failures—the single largest cost term in the table.

6.2 Cost-adjusted comparison

Translating the confusion matrices to the stated cost model (missed failure = $50,000; false alarm = $5,000): the Random Forest incurs $70 \times \$50{,}000 = \$3{,}500{,}000$ from missed detections plus $98 \times \$5{,}000 = \$490{,}000$ from false alarms, totalling $3.99M. The rolling-count logistic regression misses nothing and incurs $510 \times \$5{,}000 = \$2{,}550{,}000$ entirely from false alarms. The three baselines also miss almost nothing but raise 918–1,058 false alarms, costing $4.59M–$5.29M.

The ordering is therefore set by how many false alarms each model needs in order to reach near-perfect recall, and the rolling-count feature needs roughly half as many as the

next-best alternative.

We report this cost model as illustrative rather than universal: the \$50,000/\$5,000 ratio is a stated assumption, not a measured industrial figure, and the ranking could shift under a different ratio. The direction of the effect is worth stating explicitly, though. As the false-alarm cost rises relative to the miss cost, precision matters more and the argument in Section 6.1 strengthens; as it falls, all models converge toward the recall ordering and the distinction the paper draws becomes less consequential. A practitioner should recompute Table 4 with their own figures.

6.3 Interpretability

The interpretability argument is secondary but relevant. The health-window feature $h_{u,c} \in \{0, \ldots, w\}$ is a count of recently degraded cycles. The decision rule reduces to a threshold on a single integer, communicable to maintenance staff without ML expertise. The Random Forest's decision process across 100 trees and 21 input channels is not interpretable in the same sense. In industrial settings where operators need to understand and trust an alert before acting on it, interpretability affects adoption [1].

6.4 Selection optimism

Extending the grid of Table 4 to finer thresholds locates an interior cost minimum at $w = 3$, $\tau = 92$, costing \$1,095,000 on the seed-42 test engines—less than half the \$2.55M at the inherited setting. Reporting that figure would be indefensible, because it was obtained by minimising cost on the same engines used to report it.

Evaluating that identical (w, τ) across 30 splits gives \$1,885,167 $\pm$ \$289,063. The single-split figure was therefore optimistic by **\$790,167**, or 42%. Under the nested protocol, where (w, τ) is chosen on validation engines that are never scored, mean validation cost is \$1,526,000 against a test cost of \$2,107,667—a residual optimism of \$581,667 attributable to selection on a finite validation sample.

We report these two numbers because a cost figure obtained by grid search on a test set is a common and largely invisible failure in applied predictive-maintenance work, and because the magnitude here—a 42% understatement—is large enough to reverse a deployment decision.

6.5 Tuning is not always worth it

A secondary observation follows from the same experiment. Selecting (w, τ) per split on 20 validation engines yields a mean test cost of \$2,107,667 $\pm$ \$386,850, whereas holding (w, τ) fixed at $(3, 92)$ yields \$1,885,167 $\pm$ \$289,063—cheaper on average and with 25% lower variance. Refitting on the combined 80 train-plus-validation engines did not help either (\$2,128,000).

The selection distribution explains why: across 30 splits, 12 distinct (w, τ) pairs are chosen, the modal pair appearing in only 5. The optimum is not identifiable from 20 validation engines, so per-split tuning injects variance without locating anything.

This comparison is not clean, and we flag it rather than lean on it: the fixed pair $(3, 92)$ was itself identified by inspecting seed-42 test performance, so its advantage is partly borrowed from the same optimism quantified above. The defensible claim is the weaker one—that with a validation set this small, tuning cannot be assumed to beat a sensible fixed default, and should be checked rather than presumed.

6.6 Limitations

This study has several limitations that constrain the generality of its conclusions.

First, FD001 is the simplest C-MAPSS subset—single operating condition, single fault mode, 100 engines with relatively uniform degradation trajectories. The per-unit normalisation and degradation composite that work well here may not transfer cleanly to FD002–FD004, which have multiple operating regimes requiring explicit condition clustering before health indicators can be computed. This paper does not evaluate any subset beyond FD001, and the cost-ranking result should be treated as specific to this benchmark until tested elsewhere.

Second, neither the window length w nor the percentile threshold τ is derived from physical principles; both are design choices validated empirically within this dataset. A learned threshold—derived from, say, a one-class SVM or an isolation forest on healthy-only training data—would be more principled and is left for future work.

Third, the 30-cycle at-risk horizon is a modelling assumption; different applications may require shorter or longer detection horizons, and recall at horizon 30 may not translate to recall at horizon 15 or 50.

Fourth, we do not compare against modern deep learning approaches (e.g., LSTM, transformer-based sequence models), which could potentially dominate all models evaluated here on both recall and cost. Such a comparison is left for future work and would be needed before any claim of state-of-the-art performance.

Fifth, the per-engine calibration variant used in Table 3 assumes a burn-in period of at least 40 healthy cycles is available for every new engine, which may not hold for engines with very short lifespans; the global-threshold variant in Table 1 does not require this assumption but is a less realistic deployment scenario for a genuinely new engine with no prior healthy-phase data at all.

Sixth, the stated cost model (\$50,000/\$5,000) is an illustrative assumption, not a measured cost from a real deployment. The ranking in Table 1 could change under a different ratio, and we have not swept it systematically.

Seventh, and specific to this version: the degradation-direction assignment used in versions 1–3 placed three of fourteen sensors on the wrong side of Equation 3 (Section 4.1). Every result in those versions was produced by a partially

self-cancelling composite. The error was found by verifying the assignment against the data rather than by any reviewer, and correcting it improved the model on every metric reported here. We note it because it means the earlier versions' numbers should not be cited, and because the assignment in a hand-built health indicator is exactly the kind of assumption that is easy to state and rarely checked.

Eighth, the cost surface in Table 4 becomes unstable at high thresholds. In the extended grid, cost is non-monotonic in τ above 92—at $w = 2$, $\tau = 97$ costs \$3.23M while $\tau = 98$ costs \$2.13M. This is expected when the threshold is estimated from the sparse upper tail of the healthy score distribution, and it means operating points in that region are fragile and should not be selected without a much larger calibration sample.

7 Conclusion

On NASA C-MAPSS FD001, four of five evaluated models reach at-risk recall between 0.9984 and 1.0000 while their deployment costs span \$2.55M to \$5.29M. Recall does not distinguish them. Weighted F1 ranks the Random Forest first while it misses 70 failures and costs 56% more than the cheapest model. Neither metric, used alone, selects the model a practitioner should deploy.

Under a nested unit-level protocol—60 training, 20 validation and 20 test engines per split, with (w, τ) chosen on validation engines only—a logistic regression on a single rolling degraded-state counter costs \$2.11M $\pm$ \$0.39M against \$3.84M $\pm$ \$0.95M for a 100-tree Random Forest on all 21 raw channels, winning on cost in 30 of 30 splits at roughly 40% of the ensemble's variance.

Two selection effects are worth reporting in their own right. Choosing (w, τ) by minimising cost on the test engines understates true cost by \$790,167, or 42%. And per-split tuning on 20 validation engines is not better than a fixed setting, because 12 distinct (w, τ) pairs are selected across 30 splits and the optimum is not identifiable from a sample that size.

This version also corrects a degradation-direction error present in versions 1–3, in which three of fourteen sensors were assigned to the wrong side of the composite score. Correcting it improved every reported metric, and it is recorded here because the assignment is precisely the kind of domain assumption that is stated once and rarely verified.

The full pipeline is available on GitHub under the MIT licence. Running `corrected_pipeline.py`, `run_prompts_5_6.py` and `nested_cv.py` reproduces every number in this paper, including the nested protocol and the selection analyses of Sections 6.4 and 6.5.

Version History

This is version 4 of this record. Versions 1–2 contained a look-ahead bias in the degradation-threshold estimation; version 3 corrected it and added the three baselines reported here. Version 4 corrects a degradation-direction error affecting three sensors (Section 4.1), which required re-running every experiment; replaces the single-split cost estimate with a nested 60/20/20 protocol; adds the selection-optimism and tuning-instability analyses of Sections 6.4 and 6.5; corrects the minority-class percentage and the author affiliation; and removes one citation that could not be verified. All superseded versions remain publicly accessible on Zenodo and should not be cited.

Code Availability

All code and the preprocessing pipeline are available at: `https://github.com/Puru2001pandey/industrial-telemetry-analytics-research` The NASA C-MAPSS FD001 training file is *not* redistributed in this repository; it must be obtained directly from NASA. See the repository README for instructions.

To reproduce all results: place `train_FD001.txt` in `data/`, then run `corrected_pipeline.py` for the primary result and sensitivity grid, `run_prompts_5_6.py` for the confusion matrices and cost analysis, and `nested_cv.py` for the cost grid (Table 4), the nested protocol (Table 5) and the selection analyses (Sections 6.4–6.5). Each writes its output to `results/`.

Acknowledgements

The author thanks the NASA Ames Prognostics Center of Excellence for making the C-MAPSS dataset publicly available. This work was conducted independently, without institutional funding.

References

[1] Jardine, A.K.S., Lin, D., and Banjevic, D. (2006). A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7):1483–1510.

[2] Saxena, A., Goebel, K., Simon, D., and Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. In *Proc. Int. Conf. Prognostics and Health Management (PHM)*, Denver, CO.

[3] Ramasso, E. and Saxena, A. (2014). Performance benchmarking and analysis of prognostic methods for CMAPSS datasets. *Int. J. Prognostics and Health Management*, 5(2):1–15.

[4] Li, X., Ding, Q., and Sun, J.-Q. (2018). Remaining useful life estimation in prognostics using deep convolution neural networks. *Reliability Engineering & System Safety*, 172:1–11.

[5] Khelif, R., Chebel-Morello, B., Malinowski, S., Laajili, E., Fnaiech, F., and Zerhouni, N. (2017). Direct remaining useful life estimation based on support vector regression. *IEEE Trans. Industrial Electronics*, 64(3):2276–2285. DOI: 10.1109/TIE.2016.2623260.

[6] Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal loss for dense object detection. In *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, Venice, Italy, 2999–3007.